

Node.js Introduction

Section: 1 of 11 | Duration: 6 – 8 hours | [Theory](#) [Practical](#)

Topics covered: What is Node.js · Types of Servers · The Request Journey · V8 Engine · Event-Driven Architecture · Browser Events vs EventEmitter · Node.js vs Browser JS · Backend Tech Stack Comparison · Use Cases · Installation with NVM

Learning Objectives

- Understand what Node.js is and what problem it solves.
- Explain the V8 engine and its role in executing JavaScript on the server.
- Describe the event-driven, non-blocking I/O model at a conceptual level.
- Distinguish between the Node.js environment and the browser JavaScript environment.
- Identify real-world use cases where Node.js excels.
- Install Node.js using NVM and verify the environment setup.

1. What is Node.js?

Node.js is a **runtime environment** that allows JavaScript to run *outside the browser*, directly on a computer or server. Before Node.js existed (introduced in 2009 by Ryan Dahl), JavaScript could only run inside a browser window. Node.js changed that.

Formal definition: Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's **V8 JavaScript engine**.

It is important to understand three things Node.js is *not*:

- It is **not a programming language** – you still write JavaScript.
- It is **not a framework** – it is a runtime. Express.js (covered later) is the framework.

- It is **not a server by itself** – it gives you the tools to *build* a server.

Think of it this way: Python has the Python interpreter to run `.py` files. Node.js is the interpreter for running `.js` files on your machine.

1.1 Types of Servers

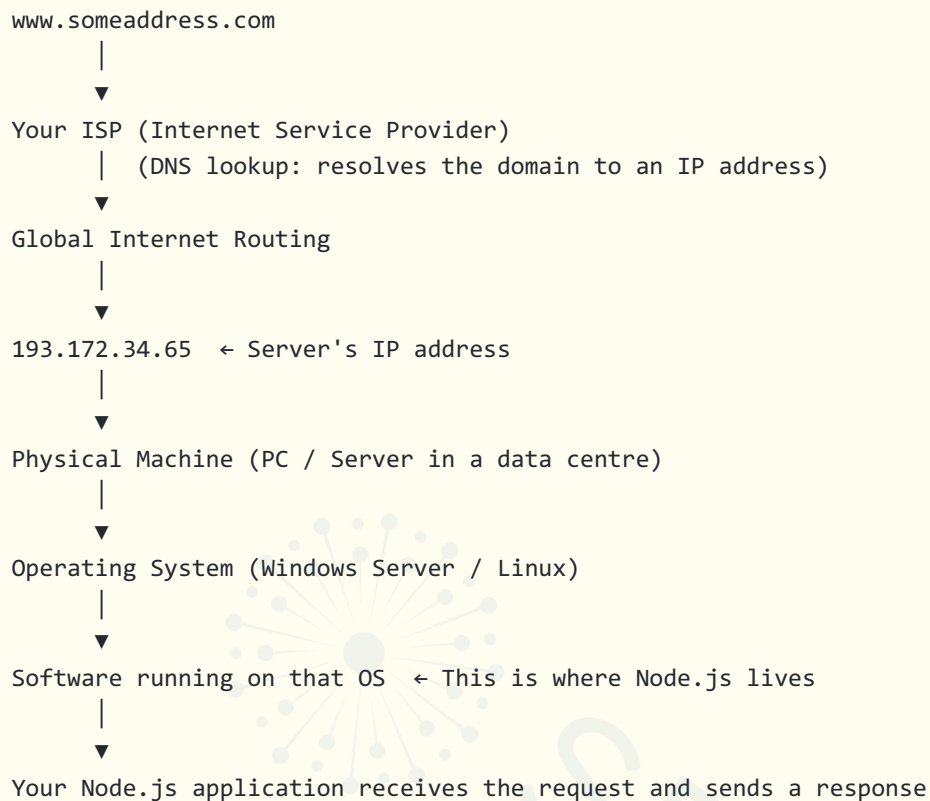
A **server** is just a computer (or a program running on one) that waits for requests and responds to them. Different types of servers are built for different jobs:

Server Type	What it does	Example software
Web Server	Responds to HTTP/HTTPS requests with HTML pages or API data	Node.js, Apache, Nginx, IIS
File Server	Stores and serves files to clients over a network	Samba, FTP servers
Chat / Real-Time Server	Maintains persistent connections and pushes messages instantly	Node.js + Socket.IO
Email Server	Handles sending, receiving, and storing emails (SMTP/IMAP)	Postfix, Microsoft Exchange
Game Server	Manages game state, player connections, and real-time updates	Custom (often C++, Node.js for lighter games)

Node.js is primarily used as a **web server** (handling HTTP APIs) and a **real-time server** (chat, notifications). It is not suited for email or game servers that require heavy CPU processing.

The Big Picture: How does a URL reach Node.js?

When you type `www.someaddress.com` in your browser, here is the full journey your request takes:



This is why Node.js needs to listen on a specific

port

(e.g., port 3000 or 443). The OS routes incoming network traffic on that port to your running Node.js process.

1.2 The V8 Engine

V8 is the JavaScript engine originally built by Google for the Chrome browser. Its job is to take JavaScript source code (which humans write) and convert it into low-level **machine code** that the CPU can execute directly.

Analogy

Think of JavaScript as instructions written in English, and your CPU as a machine that only understands binary (0s and 1s). The V8 engine is the translator in between. It reads your JavaScript and converts it into something the machine understands — at very high speed.

V8 uses a technique called **Just-In-Time (JIT) compilation**. Instead of interpreting your code line-by-line slowly, or pre-compiling everything upfront, it compiles code during execution — making it extremely fast.

Ryan Dahl extracted V8 from Chrome and wrapped it with additional capabilities (file system access, networking, etc.) to create Node.js. The key insight was: *if V8 can run JS fast in a browser, why not use it to run JS on a server?*

Component	What it does
V8 Engine	Compiles and executes JavaScript code
libuv	Handles asynchronous I/O, event loop, thread pool
Node.js Core APIs	File system, HTTP, streams, crypto, etc.

You do not need to interact with V8 or libuv directly. They work under the hood. But knowing they exist helps you understand *why* Node.js is fast and capable of handling many tasks simultaneously.

1.3 Event-Driven Architecture & Non-Blocking I/O

This is the most important concept to understand about Node.js, and it is what makes it different from traditional server environments like Apache (PHP) or Django (Python).

The Traditional (Blocking) Model

In a traditional server, every incoming request gets its own thread. A thread is like an employee who works on one task at a time. If that task involves waiting — for example, reading from a database — the employee (thread) just sits and waits, doing nothing.

Analogy – The Restaurant

Imagine a restaurant where *one waiter is assigned to one customer for the entire meal*. The waiter stands next to the customer, places the order, then waits in the kitchen until the food is ready, then brings it out. While waiting in the kitchen, the waiter cannot serve anyone else. If 100 customers arrive, you need 100 waiters. This is expensive and does not scale well.

The Node.js (Non-Blocking) Model

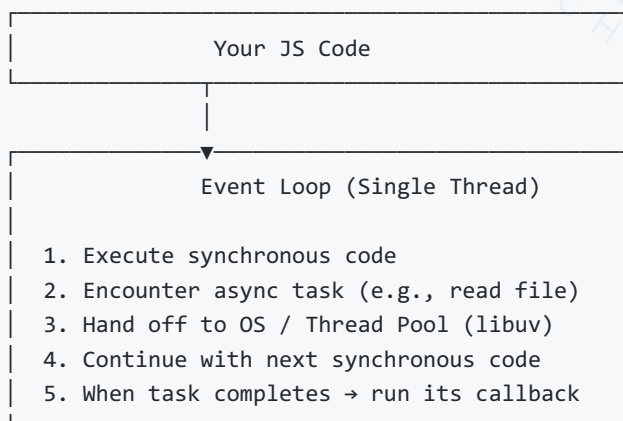
Node.js runs on a **single thread**. But instead of waiting, it uses an **event loop**. When a task involves waiting (e.g., reading a file, querying a database, calling an API), Node.js registers a *callback function* and immediately moves on to the next task. When the waiting task finishes, it fires an event, and the callback function is executed.

Analogy – The Better Restaurant

Now imagine a waiter who takes an order, gives it to the kitchen, and immediately goes to take the next customer's order. When the kitchen calls out "Order ready!", the waiter picks it up and delivers it. One waiter can now serve many customers because they are never just standing still and waiting. This is the Node.js model.

This design is called **non-blocking I/O** (I/O = Input/Output — things like reading files, network calls, database queries). It allows Node.js to handle thousands of concurrent connections with minimal overhead.

Visual Overview of the Event Loop



Note: The event loop is covered in depth in Section 3. This introduction provides just enough context to understand why Node.js is designed the way it is.

Browser Events vs. Node.js Events

In the browser you use `addEventListener` to react to user or DOM events:

```
// Browser JavaScript – DOM events
document.getElementById('btn').addEventListener('click', () => {
  console.log('Button clicked!');
});
```

Node.js has no DOM, so there are no click or keyboard events. Instead, Node.js uses the **EventEmitter** class (built into the `events` module) to implement the same idea for server-side events — things like a file finishing a read, an HTTP request arriving, or a stream ending:

```
// Node.js – EventEmitter (server-side events)
const EventEmitter = require('events');

const emitter = new EventEmitter();

// Register a listener (like addEventListener)
emitter.on('greet', (name) => {
  console.log(`Hello, ${name}!`);
});

// Emit / trigger the event
emitter.emit('greet', 'Alice');
// Output: Hello, Alice!
```

The concept is identical — *listen for an event, run a callback* — but the events are server-side (data received, file read, connection opened) rather than user interactions. The HTTP server built into Node.js is itself an EventEmitter: it emits a `'request'` event every time a browser contacts it.

2. Node.js vs. Browser JavaScript

JavaScript is the language in both environments, but what you can *do* with it is very different. The environment determines what APIs and global objects are available.

Feature	Browser JavaScript	Node.js
Purpose	Interact with the browser and webpage	Build server-side applications
DOM	Available (<code>document</code> , <code>window</code>)	Not available — there is no webpage
File System	No access (security restriction)	Full access via <code>fs</code> module
Network	<code>fetch</code> , <code>XMLHttpRequest</code>	<code>http</code> / <code>https</code> modules, full socket control
Global Object	<code>window</code>	<code>global</code> (and <code>globalThis</code> in modern Node.js)
Module System	ES Modules (<code>import/export</code>)	CommonJS (<code>require</code>) + ES Modules
Execution	Sandboxed in browser tab	Full OS access (processes, env variables)
Typical use	UI, animations, DOM manipulation	APIs, servers, CLI tools, file processing

The most important takeaway: in **Node.js**, there is no `document` or `window`. Code like `document.getElementById('app')` will throw an error. In return, Node.js gives you powerful capabilities the browser intentionally restricts, such as reading and writing files on disk.

3. Use Cases

Node.js is not the right tool for every job. Understanding where it excels helps you make architectural decisions in real projects.

Where Node.js is a strong choice:

REST APIs & Web Servers

Node.js (typically with Express) is widely used to build RESTful APIs that serve data to frontend applications, mobile apps, or other services. Its non-blocking model makes it efficient for handling many simultaneous API requests — a very common production scenario.

Examples in production: LinkedIn's mobile backend, PayPal's API layer.

Real-Time Applications

Applications that need to push data from server to client instantly — such as chat applications, live notifications, collaborative tools, or multiplayer games — benefit greatly from Node.js. Libraries like `Socket.IO` are built specifically for Node.js to enable real-time bidirectional communication.

Examples: Slack, Trello real-time updates.

Microservices

Modern applications are often split into small, independent services that communicate over a network. Node.js is lightweight and starts quickly, making it a natural fit for individual microservices in a larger distributed system.

CLI Tools & Scripting

Many developer tools are built with Node.js — including `npm` itself, Webpack, ESLint, and create-react-app. Its access to the file system, processes, and shell commands makes it practical for build tools and automation scripts.

Streaming & File Processing

Node.js has a built-in streams API that is efficient for processing large files — for instance, uploading a video or reading a large CSV — without loading the entire file into memory.

Node.js Among Other Backend Technologies

When starting a backend project, teams choose a technology stack. Here is how Node.js compares to the two other dominant enterprise stacks — Java and .NET — so you understand where it fits in the wider ecosystem:

Stack	Language(s)	Key Frameworks / Libraries	Typical Use
Java	Java	Spring (web framework), Hibernate (ORM / database)	Large enterprise applications, banking, Android backends. Strong multi-threading for CPU-heavy work.
.NET	C#	ASP.NET MVC (web), Web API (REST), LINQ, Entity Framework (ORM)	Microsoft-ecosystem enterprises, Windows Server deployments. Strong tooling with Visual Studio.
Node.js	JavaScript / TypeScript	Express, Fastify, NestJS (web); Prisma, Sequelize (ORM); Angular / React on the front-end	APIs, real-time apps, microservices, startups. One language (JS/TS) across the entire stack.

The full-stack advantage: One of Node.js's biggest selling points is that a developer who knows JavaScript can work on both the browser (Angular, React) and the server (Node.js/Express) without switching languages. This reduces context-switching and makes small teams more productive.

Where Node.js is *not* the best choice:

Scenario	Why Node.js struggles	Better alternative
CPU-heavy computation (e.g., video encoding, machine learning)	Blocks the single thread; no true parallelism	Python, Go, Java
Heavy number-crunching / scientific computing	JS is not optimised for matrix math, etc.	Python (NumPy), R

Key Insight: Node.js is excellent for tasks that are *I/O-bound* (waiting on network/file/database). It is not suited for tasks that are *CPU-bound* (heavy calculations). Knowing this distinction will come up in interviews.

4. Installation: Node.js & NVM

4.1 What is NVM?

NVM stands for **Node Version Manager**. It is a command-line tool that lets you install and switch between multiple versions of Node.js on the same machine. This is critical in professional development because different projects may require different Node.js versions.

Analogy

Think of NVM as a TV remote that controls multiple TV inputs. Instead of having one TV locked to one channel (one version of Node.js), NVM lets you switch to whichever version a project needs — without uninstalling and reinstalling Node.js manually.

Without NVM, if you upgrade Node.js globally to a newer version, older projects might break. NVM solves this cleanly.

4.2 Understanding LTS vs Current

The Node.js website offers two download tracks:

- **LTS (Long-Term Support):** Stable, recommended for production and learning. Receives bug fixes and security patches for years.
- **Current:** Latest features, but less stable. Not recommended for production systems.

Always install the LTS version when building applications or learning. As of writing, LTS is Node.js v20.x.

4.3 Installing NVM

macOS / Linux:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

After running the installer, restart your terminal, then verify:

```
nvm --version
```

Windows: Use [nvm-windows](#) — download the installer from the releases page.

4.4 Installing Node.js via NVM

Once NVM is installed:

```
# Install the latest LTS version
nvm install --lts

# OR install a specific version (e.g., 20.x)
nvm install 20

# Set a version as your default
nvm alias default 20

# Switch to a specific version for the current session
nvm use 20
```

```
# List all installed versions
nvm ls
```

Verify the installation was successful:

```
node --version
# Expected: v20.x.x (or similar LTS version)

npm --version
# Expected: 10.x.x (npm is installed alongside Node.js)
```

Note: `npm` (Node Package Manager) is automatically installed when you install Node.js. It is used to install third-party libraries. It is covered in full in Section 2.

4.5 Alternative: Installing without NVM

If you prefer a simpler setup, you can download the Node.js installer directly from nodejs.org and install it like any other application. However, NVM is the professional standard and is strongly recommended.

5. Running Node.js: Your First Scripts

With Node.js installed, you can immediately run JavaScript files from the terminal using the `node` command. This section walks through three progressive examples, each introducing a new capability.

Example 1 — Hello World

Create a file named `hello.js` and write the following:

```
hello.js
```

```
// This is our first Node.js script.  
// console.log works just like in the browser.  
  
console.log("Hello from Node.js!");  
console.log("The current date is:", new Date().toLocaleDateString());
```

Run it from your terminal:

```
node hello.js
```

Output:

```
Hello from Node.js!  
The current date is: 16/02/2026
```

What is happening? The `node` command invokes the Node.js runtime, which reads your file, feeds it into the V8 engine, and executes it. There is no browser, no HTML file, no `<script>` tag — just JavaScript running directly.

Example 2 — The `process` Object: Command-Line Arguments

Node.js exposes a global object called `process` that gives you information about the currently running Node.js process. One of its most used properties is `process.argv`, which holds the command-line arguments passed to your script.

`process.argv` is always an array. The first two elements are fixed:

- `process.argv[0]` — Path to the `node` executable
- `process.argv[1]` — Path to your script file
- `process.argv[2]` onwards — Arguments *you* pass in

```
greet.js
```

```
// process is a global object – no import needed.  
  
// Extract the name argument (index 2 onwards)  
const args = process.argv.slice(2); // removes the first two fixed entries
```

```
const name = args[0];           // first argument the user provides

if (!name) {
  console.log("Usage: node greet.js ");
  process.exit(1);  // exit with error code 1
}

console.log(`Hello, ${name}! Welcome to Node.js.`);
console.log(`Node.js version: ${process.version}`);
console.log(`Platform: ${process.platform}`);
```

```
node greet.js Alice
```

Output:

```
Hello, Alice! Welcome to Node.js.
Node.js version: v20.11.0
Platform: linux
```

Try running it without a name to see the error handling in action:

```
node greet.js
```

Output:

```
Usage: node greet.js <your-name>
```

What is happening? `process.exit(1)` terminates the script immediately. Exit code `0` means success; any non-zero code signals an error to the operating system. This is a universal convention in command-line programs, not just Node.js.

Example 3 — Environment Variables & Uncaught Exception Handling

`process.env` is an object that gives you access to **all environment variables** currently available in the OS shell session. This includes both system-level variables set by the operating system itself, and any custom variables you define.

You can inspect common OS variables like this:

env-explore.js

```
// Every OS has environment variables – process.env exposes them all.
// Note: variable names are case-insensitive on Windows, but case-sensitive on macOS/Linux.

console.log("Path:                ", process.env.PATH);           // directories the OS searches
                           for executables
console.log("Temp folder:         ", process.env.TEMP);           // Windows temp directory
console.log("Home folder:        ", process.env.HOMEPATH);        // current user's home path
                           (Windows)
console.log("Current user:        ", process.env.USERNAME);       // logged-in username (Windows)
console.log("User settings folder:", process.env.USERPROFILE);    // e.g. C:\Users\Alice
console.log("Application data:    ", process.env.APPDATA);        // roaming app data folder
                           (Windows)
console.log("Machine name:       ", process.env.COMPUTERNAME);    // hostname of this machine
```

```
node env-explore.js
```

Output:

```
Path:                C:\Windows\system32;C:\Windows;...
Temp folder:         C:\Users\Alice\AppData\Local\Temp
Home folder:         \Users\Alice
Current user:        Alice
User settings folder: C:\Users\Alice
Application data:     C:\Users\Alice\AppData\Roaming
Machine name:        ALICE-PC
```

Cross-platform note: The variables above are Windows-specific. On macOS/Linux, the equivalents are `HOME` (home folder), `USER` (username), and `TMPDIR` (temp folder). If a variable is not set on the current OS, `process.env.VARIABLE_NAME` returns `undefined` rather than throwing an error — always guard with `|| 'default'` or an `if` check in real code.

In production applications, sensitive configuration like database passwords or API keys should never be hardcoded in source files. Instead, they are stored as **environment variables** and accessed via `process.env`.

Additionally, any unhandled exception in Node.js will crash your process. The `process.on('uncaughtException', ...)` handler is a last-resort safety net.

config.js

```
// — Safety net for uncaught exceptions —————
// This should be at the top of your entry file.
// It prevents the entire process from crashing on unexpected errors.
process.on('uncaughtException', (err) => {
  console.error('An unexpected error occurred:', err.message);
  // In production, you would log this to a service (e.g., Sentry)
  // and then exit gracefully.
  process.exit(1);
});

// — Reading environment variables —————
// process.env is an object containing all environment variables
// available in the current shell session.

const appPort = process.env.PORT || 3000;    // use PORT if set, else default to 3000
const nodeEnv  = process.env.NODE_ENV || 'development';

console.log(`App Port    : ${appPort}`);
console.log(`Environment : ${nodeEnv}`);

// — Simulating a crash to test our handler —————
// Uncomment the line below to see the uncaughtException handler fire.
// throw new Error("Simulated crash!");
```

Run it normally first:

```
node config.js
```

Output:

```
App Port    : 3000
Environment : development
```

Now run it with an environment variable set inline:

```
PORT=8080 NODE_ENV=production node config.js
```

Output:

```
App Port    : 8080
Environment : production
```

What is happening? Setting `PORT=8080` before the command temporarily sets that environment variable for this single execution. `process.env.PORT` picks it up. The `||` operator provides a default fallback if the variable is not set. This pattern — `process.env.VARIABLE || defaultValue` — is extremely common in real Node.js applications.

Important: The `process.on('uncaughtException')` handler is a safety net, not a replacement for proper error handling. In most cases, errors should be handled where they occur. This handler exists to prevent silent crashes and ensure errors are at least logged before the process exits.

6. The Node.js REPL

REPL stands for **Read-Eval-Print Loop**. It is an interactive shell where you can type JavaScript expressions and see their results immediately, without writing a file. It is useful for quick experiments and debugging.

Open it by typing `node` with no arguments:

```
node
```

Output:

```
Welcome to Node.js v20.11.0.
Type ".help" for more information.
> 2 + 2
4
> const name = "Node.js"
undefined
```

```
> `Hello from ${name}`
'Hello from Node.js'
> process.version
'v20.11.0'
> .exit
```

Type `.exit` or press `Ctrl + C` twice to leave the REPL.

Summary

Concept	Key Point
Types of Servers	Web, file, chat, email, game servers each serve a different purpose. Node.js excels at web and real-time servers.
Request Journey	A URL resolves via DNS → ISP → IP → OS → your running Node.js process on a specific port.
EventEmitter	Node.js equivalent of <code>addEventListener</code> . Use <code>emitter.on(event, callback)</code> to listen for server-side events.
Backend Stacks	Java/Spring, .NET/C#, and Node.js/Express are the three dominant backend stacks. Node.js uses JS/TS across the full stack.
Node.js	A JavaScript runtime built on the V8 engine. Runs JS outside the browser.
V8 Engine	Compiles JavaScript to machine code via JIT compilation. Created by Google for Chrome.
Event-Driven / Non-Blocking	Node.js does not wait for slow tasks. It registers a callback and moves on, returning to the task when it completes.

Concept	Key Point
Node.js vs Browser	No DOM or window in Node.js. Full file system & OS access instead.
Use Cases	REST APIs, real-time apps, microservices, CLI tools. Not for CPU-heavy tasks.
NVM	Manages multiple Node.js versions. Use LTS for stability.
<code>process</code> object	Global object providing <code>argv</code> , <code>env</code> , <code>version</code> , <code>platform</code> , and lifecycle events.

Interview Questions

1. What is Node.js? Is it a programming language or a framework?

Expected answer: Node.js is a runtime environment, not a language or a framework. It allows JavaScript to run outside the browser using the V8 engine.

2. What is the V8 engine and what is its role in Node.js?

Expected answer: V8 is Google's JavaScript engine. It compiles JavaScript to machine code using JIT compilation. Node.js uses it as the core execution engine for running JS on the server.

3. Explain non-blocking I/O. How does it differ from a traditional blocking model?

Expected answer: In a blocking model, a thread waits for an I/O operation to finish before continuing. In Node.js, I/O operations are handed off and a callback is registered. The main thread continues executing other code. When the I/O completes, the callback is placed in the event queue and executed. This allows Node.js to handle many concurrent operations on a single thread.

4. What global objects exist in Node.js that do not exist in the browser, and vice versa?

Expected answer: Node.js has `process`, `global`, `__dirname`, `__filename`, and modules like `fs`, `http`. The browser has `window`, `document`, `navigator`, `localStorage`. Neither has what the other exclusively owns.

5. When would you choose Node.js over another backend technology? When would you not?

Expected answer: Choose Node.js for I/O-bound tasks: REST APIs, real-time applications (chat, notifications), microservices, and streaming. Avoid it for CPU-bound tasks like video encoding, image processing, or machine learning, where languages with better threading/multi-core support (Python, Go, Java) are more appropriate.

6. What is NVM and why is it used in professional development?

Expected answer: NVM (Node Version Manager) allows developers to install and switch between multiple Node.js versions. Different projects may require different versions. NVM avoids conflicts and lets you manage versions per project without reinstalling Node.js globally.

7. What is `process.argv`? What are the first two elements always set to?

Expected answer: `process.argv` is an array of command-line arguments. `argv[0]` is the path to the `node` executable, `argv[1]` is the path to the script file. User-provided arguments start from `argv[2]`.

Practical Assignment

Title: Build a Personal Info CLI Script

Objective: Combine everything from this section to build a small command-line script that accepts inputs and displays system and user information.

Requirements:

1. The script should accept **two command-line arguments**: `firstName` and `lastName`.
2. If either argument is missing, display a usage message and exit with code `1`.
3. If both arguments are provided, display:
 - Full name (first + last)

- Current date and time
- Node.js version (`process.version`)
- Operating system platform (`process.platform`)
- An environment variable called `MY_APP_ENV` ; if not set, display `"not set"`

4. Add an `uncaughtException` handler that logs: `"Critical error: [error message]"` before exiting.

Expected usage:

```
MY_APP_ENV=staging node info.js John Doe
```

Expected output (example):

Output:

```
— Personal Info —
Full Name   : John Doe
Date / Time  : 2/16/2026, 10:30:00 AM
Node.js     : v20.11.0
Platform    : linux
App Env     : staging
```

Bonus challenge: Accept an optional third argument `--json` . If provided, output all the information as a formatted JSON object instead of the table layout above.